Statemind Fellowship Report

11-05-2026 - 31-05-2026

Table of contents

1. Project brief

Title	Description
Project name	sw-meta-contracts-audit
Timeline	11-05-2026 – 31-05-2026

Short overview

The audited codebase is a fork of StakeWise protocol prepared specifically for the Statemind fellowship program. StakeWise is a liquid staking protocol on Ethereum: depositors send ETH to a vault that runs validators on their behalf and accrues staking and MEV rewards, and can additionally mint osToken, an over-collateralized liquid staking token, against their staked position. Meta vaults extend this model by letting a single deposit be spread across multiple sub-vaults, so users can diversify their stake across different operators and vault configurations without managing each position individually.

Project scope

The audit covered the following files:

- contracts/vaults/SubVaultsRegistry.sol
- contracts/tokens/OsTokenRedeemer.sol
- contracts/vaults/modules/VaultState.sol
- contracts/vaults/ethereum/EthMetaVault.sol
- contracts/vaults/modules/VaultOsToken.sol
- contracts/vaults/modules/VaultEnterExit.sol
- contracts/vaults/modules/VaultSubVaults.sol
- contracts/curators/BalancedCurator.sol
- contracts/vaults/modules/VaultFee.sol
- contracts/vaults/VaultsRegistry.sol
- contracts/vaults/modules/VaultVersion.sol
- contracts/vaults/modules/VaultAdmin.sol
- contracts/tokens/EthOsTokenRedeemer.sol
- contracts/vaults/modules/VaultImmutables.sol

2. Finding severity breakdown

All vulnerabilities discovered during the audit are classified based on their potential severity and have the following classification:

Severity	Description
Critical	Bugs leading to assets theft, fund access locking, or any other loss of funds to be transferred to any party.
High	Bugs that can trigger a contract failure. Further recovery is possible only by manual modification of the contract state or replacement.
Medium	Bugs that can break the intended contract logic or expose it to DoS attacks, but do not cause direct loss of funds.
Informational	Bugs that do not have a significant immediate impact and could be easily fixed.

3. Summary of findings

Severity	# of Findings
Critical	2
High	0
Medium	2
Informational	6
Total	10

4. Conclusion

During the audit of the codebase, 10 issues were found in total:

- 2 critical severity issues
- 2 medium severity issues
- 6 informational severity issues

5. Findings report

CRITICAL-01 `VaultOsToken.burnOsToken` does not sync position fee before reducing shares

Description

Lines: VaultOsToken.sol#L72

`_syncPositionFee` is invoked on every interaction with a user's position, except in `burnOsToken`. The function recalculates the user's debt by applying the accrued protocol fee, so the amount the user owes grows over time after minting. Because this update is skipped in `burnOsToken`, a user can mint osToken and later burn it without paying the accrued fee. The loss to the protocol equals the unpaid treasury fee that should have been added to the user's debt at the moment of burn.

PoC

The behaviour is reproduced by the test at `test/Critical01.t.sol`. Run with:

```
forge test --match-test test_burnSkipsFeeSync -vvv
```

Recommendation

Call `_syncPositionFee` before updating osToken position.

CRITICAL-02 Weak `OsTokenRedeemer._isMetaVault` check lets any caller drain osToken deposited in the exit queue via `redeemSubVaultOsToken`

Description

Lines: OsTokenRedeemer.sol#L468-L470

There is a method `_isMetaVault` for access control:

```
/**
* @dev Internal function to check whether the caller is a meta vault
* @param vault The address of the vault to check
* @return True if the caller is a meta vault, false otherwise
*/
function _isMetaVault(address vault) private view returns (bool) {

    // must be a meta vault
    try IVaultSubVaults(vault).subVaultsRegistry() {
        return true;
    } catch {
        return false;
    }
}
```

The comment says the function is supposed to check whether the passed vault is a meta vault, but this is not actually enforced. Any smart contract that exposes a `subVaultsRegistry` function passes the check, so `redeemSubVaultOsToken` can be executed by any malicious actor.

The only intended flow for `redeemSubVaultOsToken` is an instant swap for the meta vault. There are only two ways the meta vault can get funds: either a withdrawal request via the exit queue, or redeeming sub-vault assets, where the meta vault creates osToken positions in sub-vaults and the `osTokenVaultController` mints osToken to the redeemer's address (`OsTokenRedeemer`), after which the meta vault redeems its positions and receives the funds. Once anyone can call `redeemSubVaultOsToken` , this flow is broken.

Thus, an attacker can deploy a smart contract that mocks a meta vault, deposit funds into any valid sub-vault, and use that stake as collateral to mint osToken to any address. Then they can call `redeemSubVaultOsToken` to redeem their position in the chosen sub-vault and receive the funds at their own address:

```
// redeem osToken shares from sub vault to meta vault
IVaultOsToken(subVault).redeemOsToken(osTokenShares, msg.sender, msg.sender);
```

In `redeemOsToken` the burn is performed via `_osTokenVaultController.burnShares` . The osToken is burnt from the redeemer's own balance, i.e. the osToken that users deposited via `enterExitQueue` . The

attacker's position is reduced in the sub-vault accounting, and since there are no LTV bounds the entire position can be redeemed immediately. Crucially, the osToken the attacker minted stays in their wallet. What gets burnt is the redeemer's osToken that backs the exit queue. Thus the attacker walks away both with the minted osToken and with the redeemed assets, while the exit-queue users lose the osToken backing their claims. The protocol accounting is broken as well.

PoC

The behaviour is reproduced by the test at `test/Critical02.t.sol`. Run with:

```
forge test --match-test test_drainsExitQueue -vvv
```

Recommendation

Authenticate the caller against the `VaultsRegistry` instead of the spoofable `_isMetaVault` duck-typing check:

```
function _isMetaVault(address vault) private view returns (bool) {
    if (!_vaultsRegistry.vaults(vault)) {
        return false;
    }

    // must be a meta vault
    try IVaultSubVaults(vault).subVaultsRegistry() {
        return true;
    } catch {
        return false;
    }
}
```

MEDIUM-03 `_processRedeemRequests` bounds `redeemAssets` by sub-vault liquidity instead of meta vault's stake, reverting `redeemSubVaultsAssets` with `LowLtv`

Description

Lines: SubVaultsRegistry.sol#L888-L907, VaultOsToken.sol#L132-L154

The meta vault has an emergency mechanism for obtaining ETH. It mints osToken against its own vault-shares in sub-vaults and immediately burns that osToken, with the underlying ETH being delivered straight to the meta vault's balance. The rest of the code makes it clear that the actual collateral backing the osToken mint is vault-shares. On redemption, for instance, it is precisely those vault-shares of the owner that get burned.

Inside `_processRedeemRequests` the distribution is computed to find out how much to pull from each sub-vault. The `redeemAssets` for a sub-vault is the minimum of two values: either the curator's calculation, or the sub-vault's free ETH balance. Neither of these numbers reflects how much the meta vault actually has staked in this particular sub-vault: the curator's calculation is the overall redemption demand spread evenly across active sub-vaults, and the free balance is the sub-vault's total liquidity, which includes deposits from any of its users.

Execution then proceeds down the call chain into `_mintOsToken`, where the maximum mintable osToken is computed from the meta vault's stake in that specific sub-vault. This is where the mismatch lives. Thus,

`LowLtv` can fire, the entire transaction reverts, and with it the `redeemSubVaultsAssets` flow becomes unavailable.

PoC

The behaviour is reproduced by the test at `test/Medium03.t.sol`. Run with:

```
forge test --match-test test_redeemAboveLtv_reverts -vvv
```

Recommendation

In `_processRedeemRequests`, compute the maximum mintable osToken shares from the meta vault's actual vault-shares in the sub-vault before calling `mintSubVaultOsToken`, and clamp the requested amount accordingly.

MEDIUM-04 `updateState` reverts on any negative reward delta due to addition overflow in `_processTotalAssetsDelta`

Description

Lines: VaultState.sol#L177-L181

`_processTotalAssetsDelta` handles both state-update outcomes of a harvest: rewards and penalties. A negative delta arises, for instance, from validator slashing, and the negative branch should be applied. After the negative branch has been applied, execution is expected to terminate, since the remaining body of the function is meant for the positive case, as the inline comment on line 179 (`convert assets delta as it is positive`) confirms. However, no `return;` is present at the end of the negative branch, so execution falls through into the code that treats the same negative delta as profit. Thus, on line 180, `uint256(totalAssetsDelta)` casts a negative `int256` to `uint256`. The cast produces a huge number close to the `uint256` maximum. On line 181 this huge number is added to `newTotalAssets`, the addition overflows `uint256`, and Solidity 0.8 checked arithmetic reverts the transaction.

`_processTotalAssetsDelta` is invoked from `updateState` in two places: in `VaultState` (regular vaults) and in `VaultSubVaults` (override for meta-vaults). Therefore any harvest producing a negative delta blocks `updateState`, and state updates remain impossible for as long as the delta stays negative. Thus, losses cannot be accounted for, and the exit queue cannot be processed.

PoC

The behaviour is reproduced by the test at `test/Medium04.t.sol`. Run with:

```
forge test --match-test test_revertsOnNegativeDelta -vvv
```

Recommendation

Add `return;` at the end of the negative-delta branch (VaultState.sol#L173-L176) so the function exits before reaching the positive-delta code:

```
if (totalAssetsDelta < 0) {
    ...
    if (penalty > 0) {
        _totalAssets = SafeCast.toUint128(newTotalAssets - penalty);
    }
```

```
    return;
}
```

Equivalent alternative: wrap the positive-delta code in an `else` block.

INFORMATIONAL-05 `depositToSubVaults` reverts on the first over-capacity sub-vault

Description

Sub-vaults have a capacity, i.e., the maximum amount of assets they can accept. Distribution of assets from the meta-vault happens in `depositToSubVaults`. The split between sub-vaults is computed by `BalancedCurator`, which simply divides the assets evenly. `depositToSubVaults` then tries to send the assets to each sub-vault one by one without checking whether the deposit would exceed the sub-vault's capacity. Each iteration triggers `VaultEnterExit._deposit()`, which checks that the resulting balance fits within the capacity. If the check fails, the entire transaction reverts, the assets don't land on any sub-vault, and the funds stay idle.

PoC

The behaviour is reproduced by the test at `test/Informational05.t.sol`. Run with:

```
forge test --match-test test_depositToSubVaults -vvv
```

Recommendation

Before depositing from the meta-vault, check each sub-vault's capacity and either cap the deposit amount accordingly or skip sub-vaults that are full.

INFORMATIONAL-06 Uneven exit distribution in `BalancedCurator.getExitRequests` when an ejecting vault is present

Description

Lines: BalancedCurator.sol#L77-L101

`BalancedCurator` is meant to spread assets evenly for deposits, so it is reasonable to assume that exit assets should be distributed evenly as well. However, this invariant is broken for some vaults when an ejecting vault is present. Unlike `getDeposits`, which correctly excludes the ejecting vault from the divisor, `getExitRequests` treats the total exit-vault count as if the ejecting vault were always included:

```
uint256 exitSubVaultsCount = subVaultsCount;
```

However, the ejecting vault is later skipped during distribution:

```
if (subVault == ejectingVault) {
    // no exit request for ejecting sub-vault
    unchecked {
        // cannot realistically overflow
        ++i;
    }
    continue;
}
```

The impact depends on the values involved. For some values the result is uneven redistribution: assets are withdrawn unevenly, and the first vaults in the array lose more than the others. For other values the result is suboptimal distribution: more than one loop iteration is required, leading to extra gas use.

PoC

The behaviour is reproduced by the test at `test/Informational06.t.sol`. Run with:

```
forge test --match-test test_unevenExitDistribution -vvv
```

Recommendation

Consider ejecting vault in exit distribution:

```
uint256 exitSubVaultsCount = ejectingVault != address(0) ? subVaultsCount - 1 :
subVaultsCount;
```

INFORMATIONAL-07 `EthOsTokenRedeemer.swapAssetsToOsTokenShares` silently absorbs ETH when `convertToShares` rounds to zero

Description

Lines: OsTokenRedeemer.sol#L445-L449

`swapAssetsToOsTokenShares` is `payable` and forwards `msg.value` to `_swapAssetsToOsTokenShares` as `assets`. There, `_osTokenVaultController.convertToShares` converts the incoming assets into osToken shares, and because of floor division very small inputs (e.g. 1 wei) can round down to zero shares. On the zero-shares branch the function does not process the incoming funds in any way and does not refund them either. In practice no rational caller swaps dust, so the realistic impact is negligible.

PoC

The behaviour is reproduced by the test at `test/Informational07.t.sol`. Run with:

```
forge test --match-test test_swap1WeiAbsorbsEth -vvv
```

Recommendation

Revert on the zero-shares condition instead of silently returning.

INFORMATIONAL-08 `OsTokenRedeemer.processExitQueue` and `OsTokenRedeemer.canProcessExitQueue` are not synchronized

Description

Lines: OsTokenRedeemer.sol#L129, OsTokenRedeemer.sol#L406-L415

`canProcessExitQueue` is a view-only function that is meant to report whether `processExitQueue` is ready to execute, so that the caller does not waste gas calling it when the exit queue is not ready to be processed. This implies that the readiness condition must be kept in sync between the two functions. The time check `(exitQueueTimestamp + exitQueueUpdateDelay)` is identical in both, but the condition on shares/assets diverges.

`canProcessExitQueue` returns true when `swappedShares > 0 || redeemedShares > 0`. That is, `canProcessExitQueue` only inspects shares and does not account for assets at all. However, `processExitQueue` creates a checkpoint only when `processedShares > 0 && processedAssets > 0` where `processedShares = swappedShares + redeemedShares` and `processedAssets = swappedAssets + redeemedAssets`. In other words, the function requires both shares and assets to be non-zero.

As a result, the two functions treat queue processing as possible under different conditions: `canProcessExitQueue` may return true when assets are zero, whereas in that case a call to `processExitQueue` would not create a checkpoint. This misleads the caller and confuses anyone who reads the code.

Another issue concerns the order of operations. In `processExitQueue`, all counters are zeroed first and only then the early-return check runs. This is bad practice: conditions should be checked before changing state, otherwise non-zero data can be erased without creating a checkpoint.

PoC

Proof of concept is not applied here, since the issue is rather theoretical and is a code quality concern that may become a problem in the future. For now, `processedShares` and `processedAssets` are always either both zero or both non-zero, so all the checks work correctly.

Recommendation

In `processExitQueue` move the guard above the state mutation so the counters are cleared only when a checkpoint is actually created, and align `canProcessExitQueue` to check the same condition the function requires (`processedShares > 0 && processedAssets > 0`).

INFORMATIONAL-09 Misordered caching of `metaVault` in `enterSubVaultsExitQueue`

Description

Lines: SubVaultsRegistry.sol#L426-L432

In `enterSubVaultsExitQueue`, the storage variable `metaVault` is read from storage twice. The first read happens in the access check (`if (msg.sender != metaVault)`), and the second one immediately after, where the value is cached into a local variable (`address _metaVault = metaVault`). The caching pattern is correct, but the order is wrong: the cache is created after the access check, so the check itself does not benefit from it.

PoC

Proof of concept is not applied here, since it is a code quality issue.

Recommendation

Cache `metaVault` into a local variable before the access check so that both reads share a single SLOAD.

INFORMATIONAL-10 Mistakes in comments

Description

Lines: SubVaultsRegistry.sol#L267-L268

The comment `SLOAD to memory` is misleading, since the actual operation is an external call to the Keeper contract:

```
function isStateUpdateRequired() public view override returns (bool) {
    // SLOAD to memory
    uint256 currentNonce = _getCurrentRewardsNonce();
    unchecked {
        // cannot realistically overflow
        return subVaultsRewardsNonce + 1 < currentNonce;
    }
}
```

PoC

Proof of concept is not applied here, since it is a code quality issue.

Recommendation

Update the comment to reflect the actual operation.